

# Testing Cache Memory Access Time

Mocian Andrei

October 2025

## 1 Introduction

### 1.1 Project Proposal

In modern computer systems, memory hierarchy plays a critical role in determining overall system performance. The gap between processor speed and memory latency has grown significantly, making cache memories essential for efficient data access. This project aims to analyze and quantify the differences in data transfer speeds between cache memory, main memory (RAM), and virtual memory.

Test cases will be accessible through a GUI that makes it easier for the user to run the benchmarks and compute the results.

### 1.2 Specifications

The application will be a desktop application, written in C and Python (Tkinter and Matplotlib will be used for the GUI), that will give the user the possibility to select an algorithm to test various parameters regarding cache memory, main memory, and virtual memory. The user will also be able to load their programs and check if they are accessing cache memory efficiently.

Results will be plotted on a graph in the GUI, but the user will have the ability to export the results in PDF format.

### 1.3 Objectives

#### 1. *Application-related:*

- **Reliability:** The application should not be prone to unexpected behavior.
- **User-friendly interface:** It should be clear for the user what type of benchmark is running and the results are displayed

#### 2. *Benchmark-related:*

- **Accuracy:** Inconsistencies could appear during benchmarking. Benchmark results must truly reflect cache, RAM, and paging behavior, not system noise.
- **Consistency:** Benchmark results must be stable and comparable across runs, sessions, and machines under the same configuration.
- **Efficiency:** The application and benchmarks must use CPU, memory, and I/O resources efficiently while keeping measurement overhead negligible.

### 1.4 Project plan

#### 1.4.1 2<sup>nd</sup> meeting

Project proposal, planning, and bibliographic study.

#### 1.4.2 3<sup>rd</sup> meeting

Updated documentation, analysis and initial design.

#### 1.4.3 4<sup>th</sup> meeting

Updated design based on feedback and start implementation.

#### 1.4.4 5<sup>th</sup> meeting

Implement test cases, integrate measuring methods, verify accuracy.

#### 1.4.5 6<sup>th</sup> meeting

GUI and plots, run tests on different environments.

#### 1.4.6 Final meeting

Delivery of the project.

### 1.5 Bibliographic study

#### 1.5.1 Cache memory

Cache memory in a processor is a high speed, low capacity memory. Its speed is due to the fact that it stays very close to the processor. It works by keeping copies of zones from the main memory. This memory is not visible to the programmer, and it is managed by the Memory Management Unit (MMU).

Modern CPU architectures use multiple layers of caching. There are usually 2 layers (L1 and L2) per core, and a shared L3 layer. The configuration of a cache means its size, associativity and number of lines.

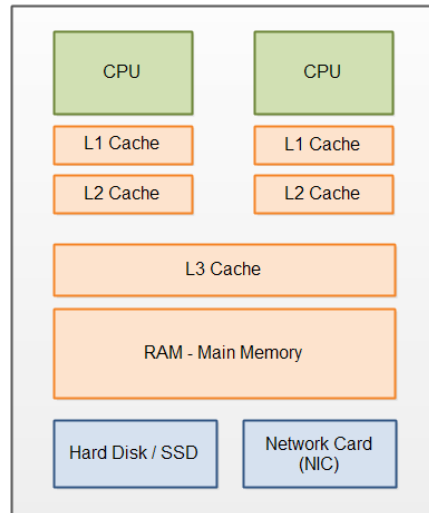


Figure 1: Cache architecture in a modern processor

If the data requested by the processor appears in the upper level it is called a **hit**. If the data is not found in the upper level, the request is called a **miss**. The lower level in the hierarchy is then accessed to retrieve the block containing requested data. In modern machines L1 is first searched for data/instruction requested by the processor. If it is a hit then that data/instruction is copied to some register in the processor. Otherwise L2 is searched. If it is a hit then data/instruction is copied to L1 and from there it is copied to a register. If the request to L2 also is a miss then the same process will repeat for L3. If the last request to L3 is a miss, the data will be searched for in the main memory.

The hit/miss rate influences the average memory access time, according to the formula:

$$t_a = t_c + (1 - R_s) t_i$$

Where:

$t_a$  - average access time

$t_i$  - access time of the internal memory

$t_c$  - access time of the cache memory

$R_s$  - success rate

$(1 - R_s)$  - miss rate

### 1.5.2 Virtual memory

Virtual memory enables the system to use disk space as an extension of RAM, at a cost of higher access time due to page faults and I/O latency. The Translation Lookaside Buffer (TLB) caches virtual-to-physical address translations, and its efficiency directly influences virtual memory performance.

### 1.5.3 Measuring hit/miss ratio

An accurate way of measuring cache misses is by using the **PMU** (Performance Measuring Unit) available on many CPUs. This unit contains registers that monitor many parameters, and can be read by the programmer.

Another way is by simulating a cache model. This method does not require access to the PMU. It is done by instrumenting the code, tracking every memory ref, and running a software cache model. This is the solution that **Valgrind** (**Cachegrind**) uses.

### 1.5.4 Cache Memory Organization

- **Direct Mapped Cache:** each block has only one corresponding place in the cache
- **Fully Associative Cache:** a block can be placed anywhere in the cache
- **Set Associative Cache:** a block can be placed in a restricted set of places in the cache. N sets in a cache  $\rightarrow$  N-way Set Associative

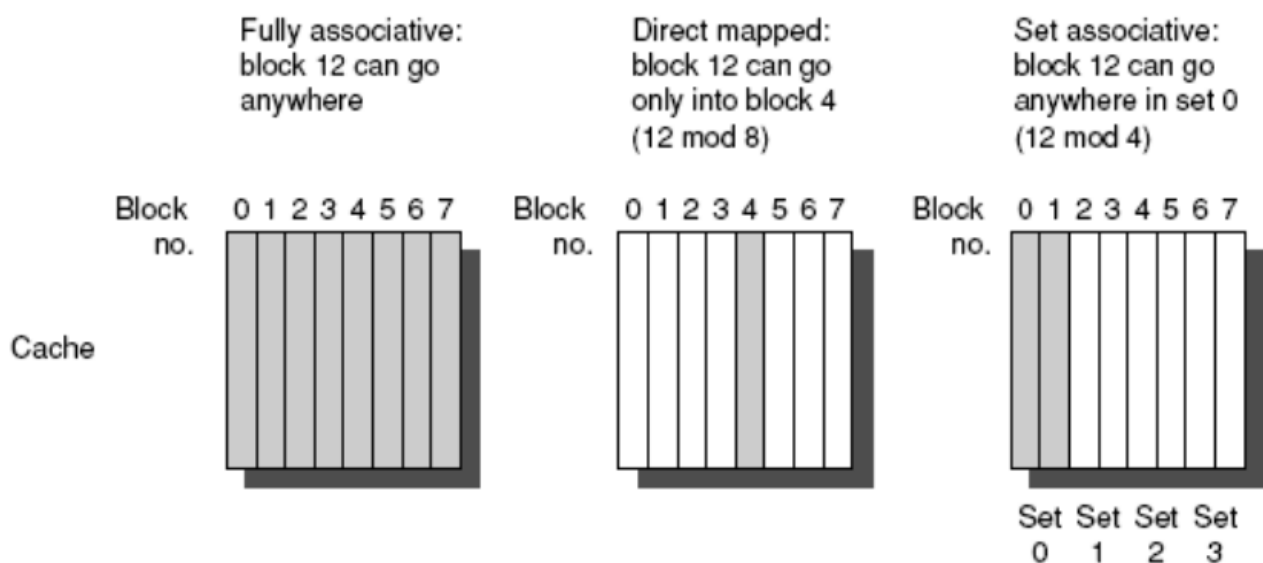
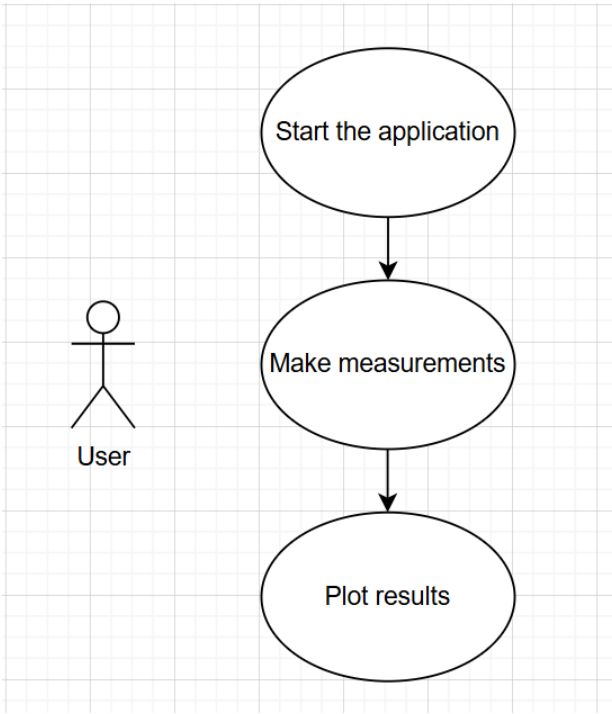
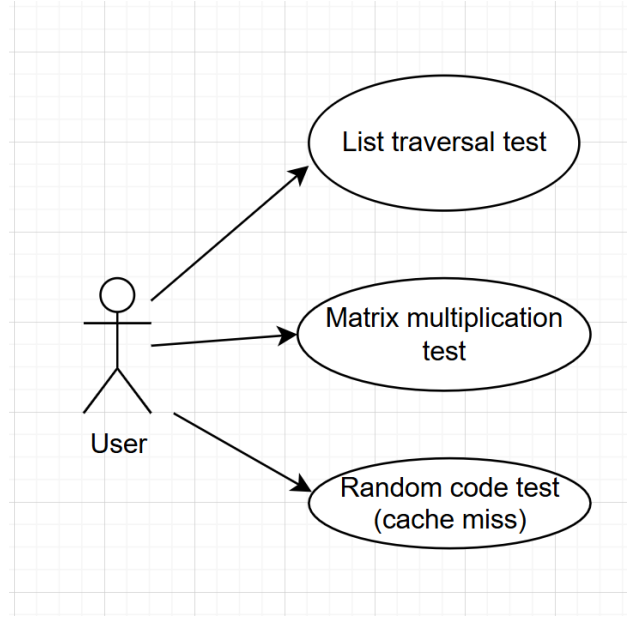


Figure 2: Cache Organization

## 2 Analysis



(a) Activity diagram



(b) Use case diagram

### 2.1 Functionalities

#### 2.1.1 List traversal test

This test is designed to measure the time required to traverse and perform other operations on a doubly linked list. It does so by gradually increasing the working set size. As the list grows, the miss ratio for L1, L2 and L3 caches will become larger, increasing the average time required to access the specific data from the memory.

The plot below shows the increase in the cycle count in relation to the working set size. As the first layer of cash fills up, the time increases, since now we have cache misses, and looking through L2 is now needed. We can see a sharp increase when accessing the data from the main memory.

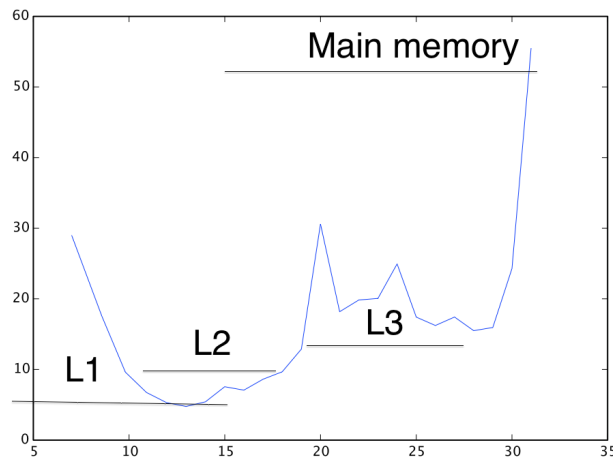


Figure 4: Cycles per access vs working-set size

### 2.2 Matrix multiplication

In programming, a 2D matrix is represented as a 1D array. A simple matrix multiplication algorithm needs 3 variables  $i$ ,  $j$ ,  $k$ , used to control the 3 loops.

The basic implementatio of the algorithm is:

```

for  $i \leftarrow 0$  to  $n - 1$  do
  for  $j \leftarrow 0$  to  $n - 1$  do
     $sum \leftarrow 0$ ;
    for  $k \leftarrow 0$  to  $n - 1$  do
       $sum \leftarrow sum + A[i \cdot n + k] \cdot B[k \cdot n + j]$ ;
    end
     $C[i \cdot n + j] \leftarrow sum$ ;
  end
end
return  $C$ 

```

**Algorithm 1:** Basic implementation of the multiplication algorithm

In order to get all the combinations possible,  $i$ ,  $j$ ,  $k$  loops can be permuted, resulting in 6 different combinations.

```

for  $j \leftarrow 0$  to  $n - 1$  do
  for  $i \leftarrow 0$  to  $n - 1$  do
     $sum \leftarrow 0$ ;
    for  $k \leftarrow 0$  to  $n - 1$  do
       $sum \leftarrow sum + A[i \cdot n + k] \cdot B[k \cdot n + j]$ ;
    end
     $C[i \cdot n + j] \leftarrow sum$ ;
  end
end
return  $C$ ;

```

**Algorithm 2:** JIK version of the algorithm

```

for  $i \leftarrow 0$  to  $n - 1$  do
  for  $k \leftarrow 0$  to  $n - 1$  do
     $aik \leftarrow A[i \cdot n + k]$ ;
    for  $j \leftarrow 0$  to  $n - 1$  do
       $C[i \cdot n + j] \leftarrow C[i \cdot n + j] + aik \cdot B[k \cdot n + j]$ ;
    end
  end
end
return  $C$ ;

```

**Algorithm 3:** IKJ version of the algorithm

And so on...

Upon closed look, we can find that 2 of the 6 implementations are always the same:

- IJK and JIK
- KIJ and IKJ
- JKI and KJI

As we discussed earlier, data is stored in 1D arrays. A matrix row is stored in consecutive buckets, also called **stride**. The problem is that moving data is not done individually, but rather line by line. An average cache line size in 64 Bytes. When an Int variable is 4 bytes, 16 elements can be transferred simultaneously.

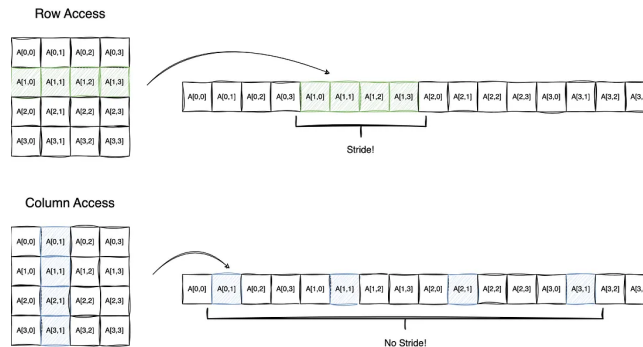


Figure 5: Row vs Columns Access

As we discussed, we can categorize the mentioned groups into 3 configurations:

- **IJK** and **JKI**

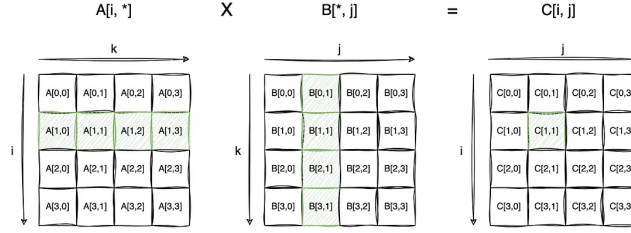


Figure 6: Inner Loop Code

```

for  $k \leftarrow 0$  to  $n - 1$  do
  |  $sum \leftarrow sum + A[i \cdot n + k] \cdot B[k \cdot n + j];$ 
end
 $C[i \cdot n + j] \leftarrow sum;$ 

```

**Algorithm 4:** Inner loop code

The sum does not need to be written in the array C until the loop is finished. Therefore, the cash does not need to be accessed for C (0). Matrix A can benefit from row stride (1/16), and matrix B needs vertical access, which leads to unfortunate access (1).

- **KIJ** and **IKJ**

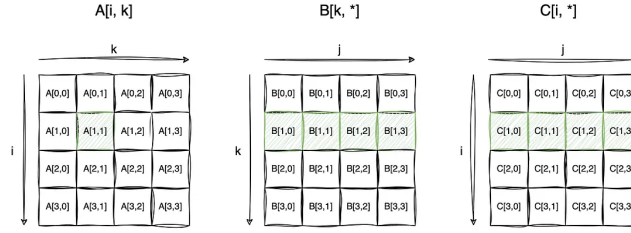


Figure 7: Most inner loop

```

for  $k \leftarrow 0$  to  $n - 1$  do
  |  $aik \leftarrow A[i \cdot n + k];$ 
  | for  $j \leftarrow 0$  to  $n - 1$  do
  | |  $C[i \cdot n + j] \leftarrow C[i \cdot n + j] + aik \cdot B[k \cdot n + j];$ 
  | end
end

```

**Algorithm 5:** Most inner loop

Matrix A is not accessed in the most inner loop, resulting 0 cache access. Matrices B and C are accessed in rows, which means that they can benefit from stride reading and writing.

- **KIJ** and **IKJ**

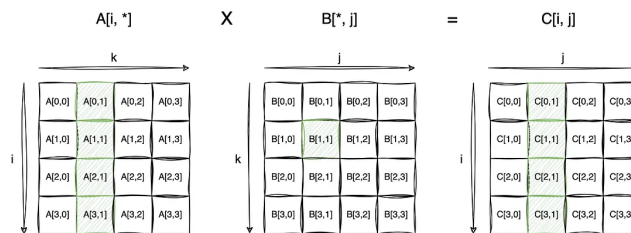


Figure 8: KIJ and IKJ

Probably the slowest solution, since we take a penalty when accessing matrix A (1), and matrix C (1).

## 3 Design

### 3.1 Overall System Design

Each type of test will be a separate module.

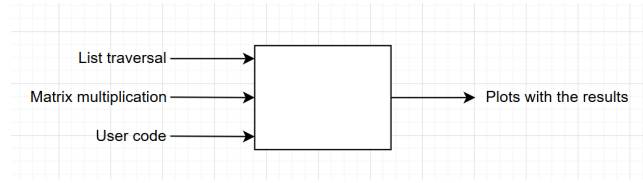


Figure 9: System black box

### 3.2 Subsystems

Each subsystem will dynamically retrieve the processor cache size using the Windows API. It queries detailed cache topology information using **GetLogicalProcessorInformationEx()** with the **RelationCache** flag. The returned data consists of a sequence of variable-length records, each describing a cache structure present in the system. The function iterates through these records using pointer arithmetic, advancing by the size of each record. For every cache descriptor encountered, the cache level (L1, L2, or L3) and its size are extracted.

#### 3.2.1 List traversal test module

This module generates a singly-linked list in 2 ways: sequentially and randomly, as seen below:

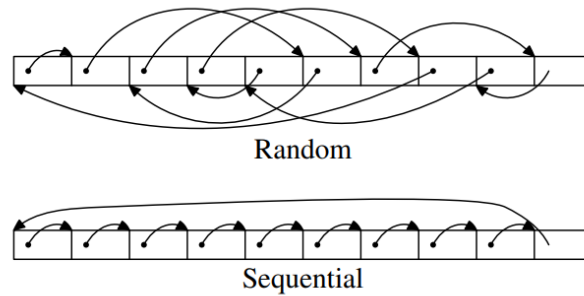


Figure 10: Test list layout

The program creates an array corresponding to the working set size of elements of this type:

```
struct l {
    struct l *n;
    long int pad[NPAD];
};
```

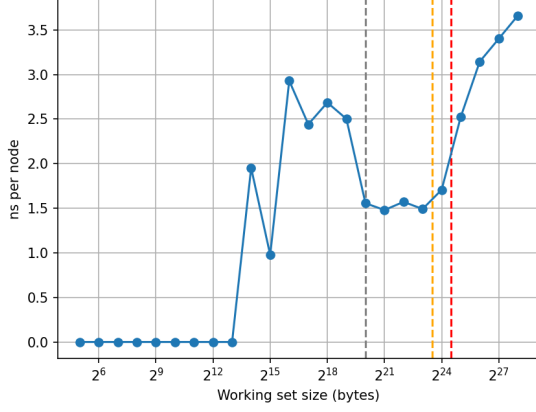
Figure 11: Working Set Element

All entries are chained in a circular list using the `n` element. In each access we do a simple addition to make sure that something is done for each node, preventing the compiler to optimize the entire loop.

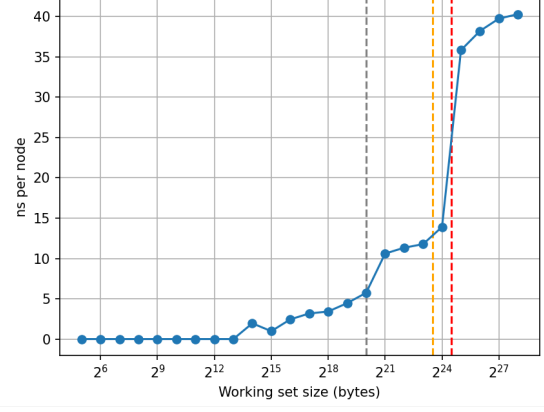
The size of a node depends on the `NPAD` variable. For this test, we will have `NPAD=7` which means that a node will have 32 bytes (half of a cache line).

To measure time, we use **QueryPerformanceCounter()** and **QueryPerformanceFrequency()**. For each size, the program does 1000 iterations for more accurate results, and then it computes the average value.

After the test is done, we can plot the results to find how the working set size impacts the time per node access.



(a) Sequential List



(b) Random List

The plots give us a better understanding of the results. On them, we can see 3 lines, each representing the memory for each layer of cache. In this experiment, I am using the I9-13900H CPU, that has 3 cache layers (L1: 1.2 MB  $\approx 2^{20}$  bytes, L2: 11.5 MB  $\approx 2^{23}$  bytes, L3 (shared): 24 MB  $\approx 2^{25}$  bytes).

When the working set is small (below  $2^{20}$  bytes), the CPU is extremely fast because all list nodes fit entirely inside the L1 cache, which has very low latency. As the working set grows, it eventually exceeds the capacity of L1, L2 (around  $2^{23}$  bytes), and finally L3 (around  $2^{25}$  bytes). Each time a cache boundary is crossed, a larger portion of the traversal must be served from a slower level of memory, producing a noticeable increase in the measured latency.

This behavior is especially visible in the **random-access list**. Unlike the sequential case, random pointer chasing destroys spatial locality. As a result, even moderately sized working sets begin to incur L2 and L3 cache misses, and once the list no longer fits into L3, almost every access becomes a DRAM access. This causes the much larger and sharper jumps in latency observed in the random-access plot. In contrast, the sequential list benefits from contiguous memory layout.

### 3.2.2 Matrix multiplication module

This module will generate random matrices with size of 1024x1024 elements. We will run the algorithm for each variation of the algorithm, to find how it impacts the running time. To measure execution time, we will use, similar to the last module, **QueryPerformanceCounter()** and **QueryPerformanceFrequency()**. The results come as expected, with the **IKJ** variation being the fastest.

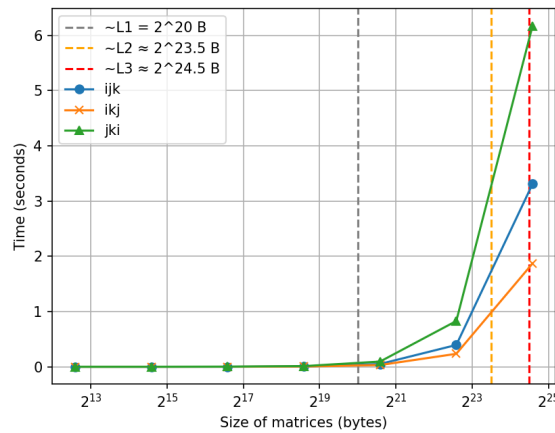


Figure 13: Matrix Multiplication Time

The graph illustrates the critical relationship between memory hierarchy and algorithm performance. For small matrix sizes that fit entirely within the fast L1 and L2 caches, execution times for all three loop orders are negligible and nearly identical. However, a sharp increase occurs as the data size exceeds the L2 and approaches the L3 cache limit. At this point, the data no longer fits in the high-speed cache and must be fetched from

the significantly slower main RAM. This "cache miss" penalty causes the execution time to grow, exposing the inefficiency of the jki loop order, which suffers from poor spatial locality, while the ikj order remains the fastest.

### 3.3 Forcing cache misses by flushing the cache lines

In this experiment, I am using a `_mm_clflush()` to flush a specific cache line containing the byte pointed to by the pointer operand. In this way I am trying to force misses on the cache memory. Since `_mm_clflush()` takes a long time, I separated the execution of the program into 2 parts: one function is doing all the work (adding and flushing), and another is just flushing the cache lines. In the end, the result is printed as the difference between these 2.

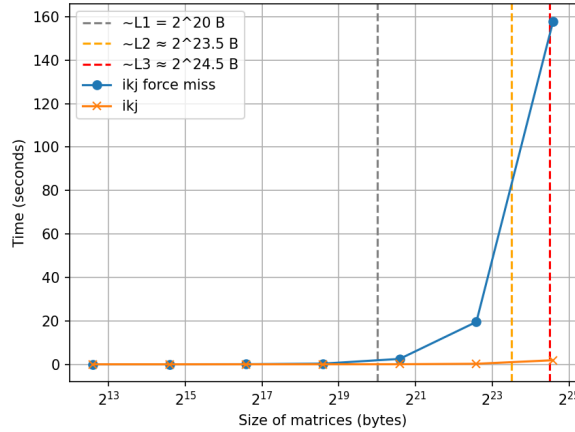


Figure 14: Forcing Misses Time

As we can see on the graph, when flushing the cache lines, the time taken to multiply the matrices is much longer, especially as the size of the matrices increases.

#### 3.3.1 GUI Module

This module contains the GUI logic. It all starts from the python main file, from where the **Program** class is run. This special class contains the **Tkinter** main loop, the main program state (which window to render), and constructors for each class representing each algorithm (this may be also called the **State Design Pattern**). The test classes contain logic to run the executables (written in C), and to plot the results onto the interface using **matplotlib**. This approach was chosen because it was convenient to have the plots embedded into the GUI.

Also this module supports generating a **PDF** after an algorithm has run and data is available.

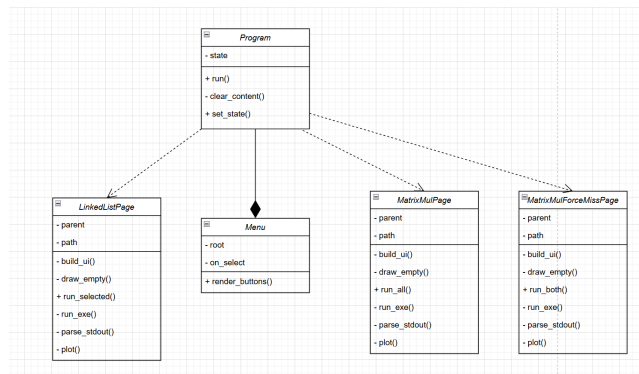


Figure 15: GUI Class Diagram

## 4 User Manual

### 4.1 Build and Run

Since this is a python application the build process is very simple. First, in the project files, there is a **requirements.txt** file, which contains the necessary packages to be installed. It is recommended that the user installs them in a local **venv**, to not interfere with the global **Python** installation. To do that, the user should run the following commands:

- `python -m venv ./venv` (this should be run only once to create the venv)
- `venv\Scripts\activate.bat`
- `pip install -r requirements.txt`
- `cd GUI`
- `python main.py`

### 4.2 Using the app

The GUI has a left sidebar, with buttons that point to all possible tests.

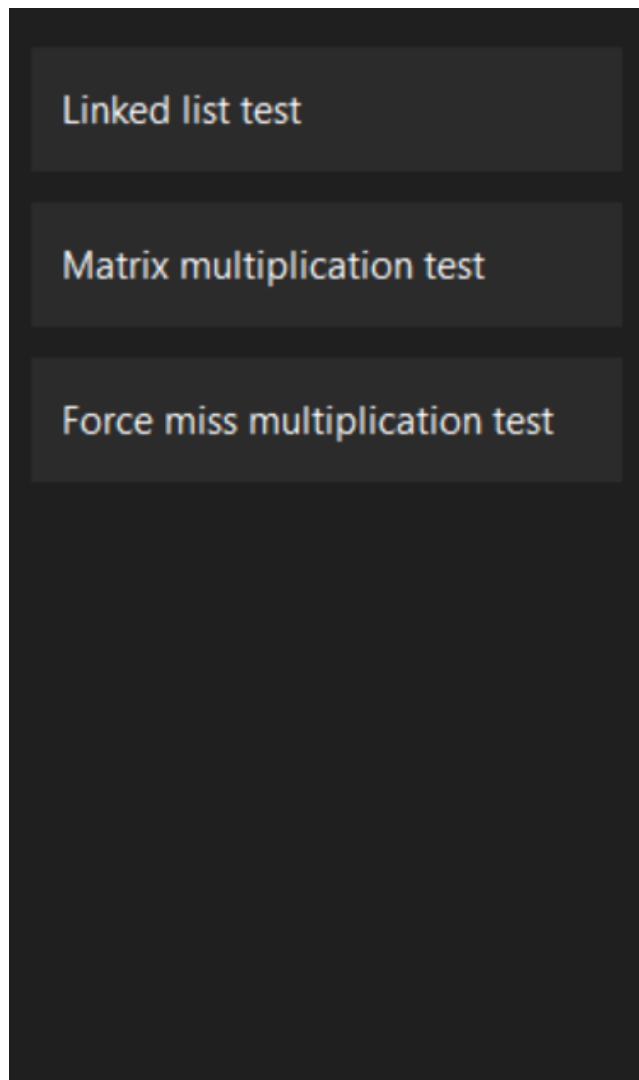
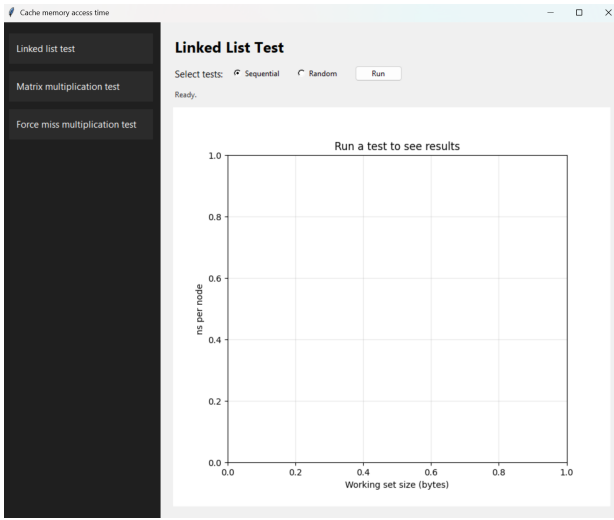
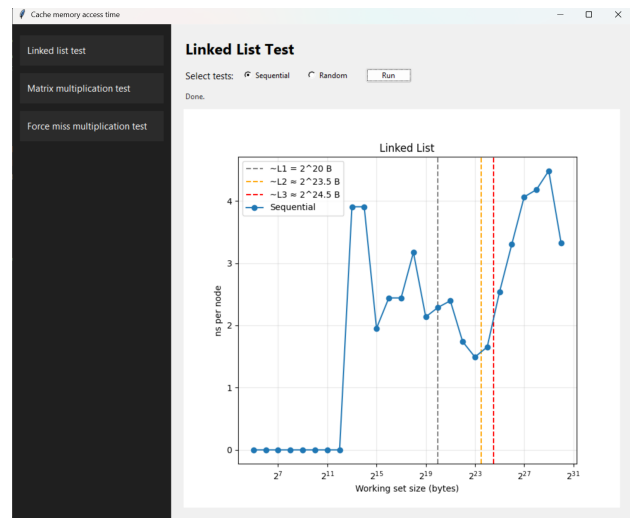


Figure 16: Sidebar

If the user selects a test, the corresponding options will appear along with an empty plot. The user selects the appropriate settings, and presses run. After the benchmark is executed, the results will appear on the plot.



(a) Before the benchmark run



(b) After the benchmark run

#### 4.2.1 Generating PDF

After a test is done, the user will have the ability to export the results to a PDF. The pdf will contain the graph, and some useful data about the test that has just run.

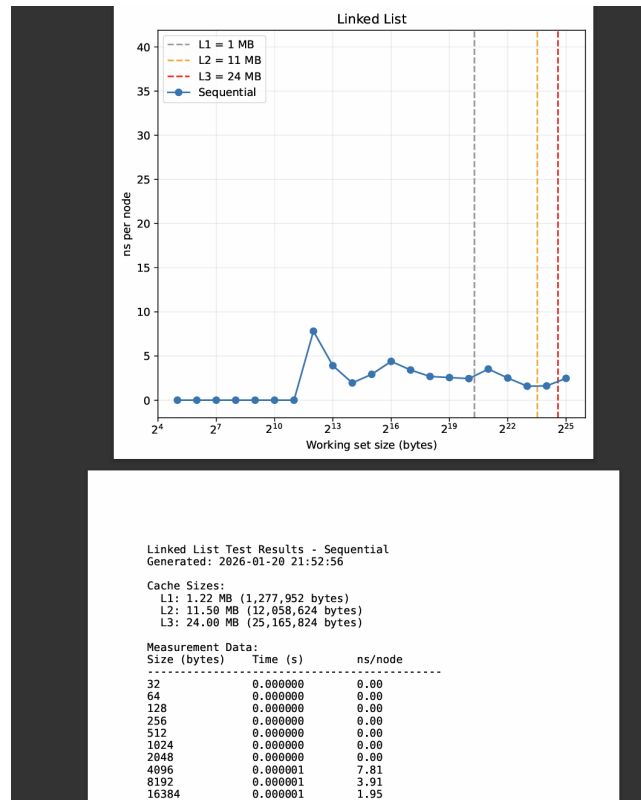


Figure 18: PDF file generated after the test is run

## 5 Bibliography

1. Structure of Computer Systems. Course 9. Memory hierarchy. [SSC\\_course\\_9\\_mem\\_hierarchy](#)
2. Why loops do matter — A Story of Matrix Multiplication, Cache Access Patterns and Java <https://medium.com/@Styp/why-loops-do-matter-a-story-of-matrix-multiplication-cache-access-patterns-and-java-859111845c25>
3. Cache Profiling and the SPEC Benchmarks: A Case Study [https://minds.wisconsin.edu/bitstream/handle/1793/9634/file\\_1.pdf?sequence=1&isAllowed=y](https://minds.wisconsin.edu/bitstream/handle/1793/9634/file_1.pdf?sequence=1&isAllowed=y)

4. Valgrind HOWTO <https://tldp.org/HOWTO/Valgrind-HOWTO/deeper.html>
5. Profiling CPU Cache/Memory from the OS/Application? <https://stackoverflow.com/questions/10122520/profiling-cpu-cache-memory-from-the-os-application>
6. Intel® Performance Counter Monitor inside your programs <https://www.intel.com/content/www/us/en/developer/articles/tool/performance-counter-monitor.html>
7. What Every Programmer Should Know About Memory  
<https://www.akkadia.org/drepper/cpumemory.pdf>
8. Tkinter Tutorial  
<https://www.geeksforgeeks.org/python/python-tkinter-tutorial/>